Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	**Master of Computer Application**	
Course	**Programming from First Principles**	
Topic Name	**The use of infinite data structures to separate control from action, Part 1.2 (practice session)**	
Topic Code	**-**	
Unit/Module No. & Name	**6**	**Laziness**
Self-Learning Hours	**-**	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4-5
2.0	Meaning And Definition	6-8
3.0	Nature Or Type	10-11
4.0	Examples And Case Studies	12-13
5.0	Keywords And Touch Vocabulary	14

OBJECTIVE

1. This lesson explains iterators in Python using simple language and an academic tone.
2. By the end of the lesson, students should be able to describe what an iterator is.
3. Students should understand why lazy evaluation is important.
4. Students should also understand how iterators help manage large or endless data.
5. Students will learn how Python uses the iterator protocol.
6. They will understand how iterators separate data production from data use.
7. They will also learn how this idea supports clean algorithm design.
8. The lesson is written for undergraduate MCA students.
9. The language is kept simple and easy to read.
10. The lesson uses short sentences and clear examples to help students understand better.

1.0 INTRODUCTION

Python is used for apps, data work, and problem solving. Many programs need to handle data that is very large, or data that never really ends. Examples include sensor readings, web requests, server logs, and live market prices. If a program tries to load all of this data into memory at one time, it may become slow or crash. Memory is limited, and copying big data also takes time. So, Python offers tools that let a program handle data step by step.

An iterator is one of these tools. An iterator is an object that gives one value at a time. The next value is made only when it is asked for. This is called lazy evaluation. Lazy evaluation means “do it later,” only when needed. This approach saves memory and often saves time too. It also supports better program design. The code that makes values can stay separate from the code that uses values. This helps students understand how loops work, and it helps engineers build systems that process streams safely.

1.1 Importance Of Iterators In Python

Iterators are important because they let Python work with big or endless data in a safe way. When data is produced one item at a time, the program does not need to store all items. This keeps memory use low. It also means that a program can stop early. If a task needs only the first 100 lines of a log file, it can stop after 100 lines. The rest of the file is never loaded and never processed.

Iterators also support reuse and flexibility. The same data producer can feed different consumers. One consumer may filter values, another may count values, and another may stop when a rule is met. This separation of data generation and consumption improves code design and reuse. It also makes testing easier, because each part has one clear job.

1.1.1 Iterators And Lazy Evaluation

Lazy evaluation is a core idea in “laziness,” the unit topic. In strict evaluation, a program computes many results right away, even if some results are never used. In lazy evaluation, the program delays work until the result is required. Iterators follow lazy evaluation by producing the next value only when next() is called or when a for loop requests it.

This delay can reduce cost. If a dataset has one million rows, but a student needs only the first ten rows to test code, an iterator can supply those ten rows without building the full list. Lazy evaluation is also vital for infinite data. Infinite data cannot be stored as a whole, because it never ends. But it can be processed if it is produced in small pieces and used in small pieces.

1.2 Iterators In Real-World Programming

Real systems often work with streams, which are flows of data over time. A stream may come from a file, a network, or a device. For example, a server log grows as the server runs. A sensor keeps sending values. A social media feed updates every second. In these cases, the total size is not fixed. Iterators help because the program can read one item, process it, and then move on. The consumer chooses a stopping point. It might stop at the end of a file, after a time limit, or after a target value is found.

1.3 Connection To Infinite Data Structures

An “infinite data structure” is a way to describe a sequence that can keep providing more values, without ever finishing. A real computer cannot store infinite items, so the structure is defined by a rule that produces the next part when asked. Iterators model this idea in Python. They do not hold an infinite list. They hold a small rule and a small state.

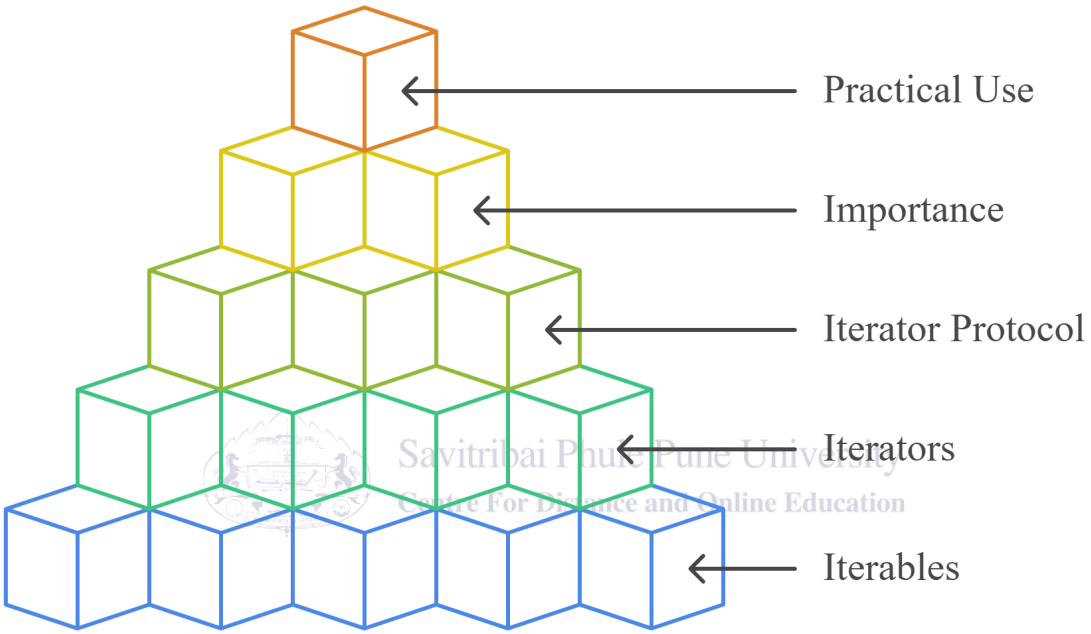
For example, an even-number iterator holds the current even number and the rule “add two.” When a consumer asks for the next value, the iterator applies the rule and returns the next even number. The action is to compute the next value. The control is to decide how long to keep asking. This is how control is separated from action, which is the key theme of this topic.

2.0 MEANING AND DEFINITION

To learn iterators well, students need both an easy meaning and a formal definition. Meaning explains how iterators are used in everyday Python programs. Definition explains the rules that make an object an iterator. Together, these parts help students reason about performance, memory, and program structure.

Fig. 1

Iterator Hierarchy



The image shows an Iterator Hierarchy arranged like stacked cubes. At the base are Iterables, followed by Iterators, then the Iterator Protocol. Above that is Importance, and at the top is Practical Use, forming a structured learning pyramid.

2.1 Meaning Of Iterators In Python

In simple terms, an iterator is a tool for moving through values one by one. It can be created from many Python objects, like lists, tuples, strings, sets, and dictionaries. These objects are called iterables, because they can be iterated over. When Python starts a for loop, it first calls iter() on the iterable to get an iterator. Then it keeps asking for the next value until there are no more values.

The meaning of an iterator is also about roles. The iterator produces values, while the loop consumes values. The loop does not need to know how the values are stored or made. It only needs to know how to request the next value.

2.1.1 Iterator Protocol In Python

Python defines a clear rule set called the iterator protocol. An object follows the protocol if it provides two special methods: iter() and next(). The iter() method returns an iterator object, and for many iterators it returns self. The next() method returns the next value. When there are no more values, next() raises StopIteration.

This protocol is how Python makes for loops work. A for loop gets an iterator, then calls next() behind the scenes. When StopIteration happens, the loop ends in a clean and predictable way. Understanding this helps students write custom iterators and avoid common mistakes.

2.2 Definition Of Iterators

Formally, an iterator is an object that follows the iterator protocol and gives values one after another when asked. It holds state, which means it remembers where it is in the sequence. Unlike a list, it does not store every value at the same time. Instead, it produces or reveals each value as iteration moves forward. This design supports memory-efficient processing.

2.2.1 Difference Between Iterable And Iterator

An iterable is any object that can return an iterator. Many built-in containers are iterable. An iterator is the object that does the step-by-step delivery. It moves forward each time you ask for the next value, and it usually cannot move backward. Most iterators are single-use. Once they are exhausted, calling next() again will keep raising StopIteration.

Every iterator is also iterable, because calling iter() on an iterator returns the iterator itself. But not every iterable is an iterator. A list is iterable, but it is not an iterator. You must call iter(list_object) to get a list iterator. This difference matters when the same iterator is shared between more than one consumer.

2.3 Importance Of Iterators In Programming

Iterators improve efficiency and program design. They allow large data to be processed without loading it all at once. This supports stable programs, because memory overload is less likely. Iterators also match many algorithm patterns: get the next item, test it, and update a result.

They also support modular design. A producer can be written once, and many consumers can reuse it. A consumer may stop after a count or after a rule is met. The action is producing the next value. The control is the rule for stopping and processing.

2.4 Using iter() And next() In Practice

Python offers iter() and next() to work with iterators directly. iter() takes an iterable and returns an iterator. next() takes an iterator and returns the next value. If the iterator is finished, next() raises StopIteration.

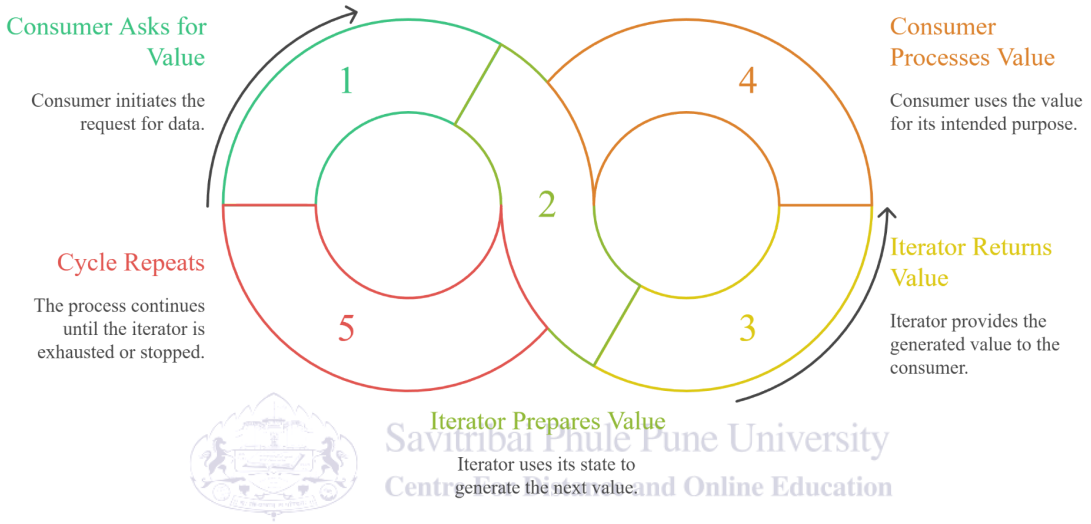
If a list is [10, 20, 30], iter() creates a list iterator. next() returns 10, then 20, then 30. A fourth call raises StopIteration. If you want to iterate again, you must call iter() again to get a new iterator. The next() function can also take a default value as a second argument, which can help avoid an exception in some safe-reading cases.

3.0 NATURE OR TYPE OF ITERATORS IN PYTHON

The nature of iterators describes how they behave. Iterators are lazy, state-based, and often single-pass. They can be finite, meaning they end, or infinite, meaning they can continue unless stopped. These traits connect directly to the topic of laziness, because laziness is about delaying work and producing values only when needed.

Fig. 2

The Iterator Cycle



The image shows the Iterator Cycle in Python. It explains how a consumer requests a value, the iterator prepares and returns the value, and the consumer processes it. The cycle repeats until the iterator is exhausted or stopped by the program.

3.1 Lazy Evaluation In Iterators

Lazy evaluation means the iterator does not do extra work. It produces one value, then waits. If the program stops early, the iterator does not waste time generating values that will never be used. This is useful when data is large and when each value is costly to compute.

3.1.1 State-Based Nature Of Iterators

Iterators keep state. State is the stored information that lets the iterator continue from where it left off. Each next() call changes the state and moves forward. Because of state, iterators are commonly exhausted and cannot restart automatically. To restart, you must create a new iterator from the original iterable or reset the state in a custom iterator class.

3.2 Finite And Infinite Iterators

Finite iterators stop after a limited number of steps. They raise StopIteration to mark the end. Infinite iterators do not have a built-in end. They keep producing values, so the consumer must apply control, such as taking only the first n values or stopping when a condition becomes true. Without control, a loop can run forever.

3.3 Separation Of Producer And Consumer

The producer-consumer separation is a key design idea. The producer is the iterator or generator that makes values. The consumer is the loop or function that uses values. The producer does not decide how many values will be used. The consumer does. Control sits with the consumer, and action sits with the producer. This improves modularity, because the same producer can serve many different consumers.

3.4 Iterators And Memory Efficiency

Iterators reduce memory use because they do not store all values. Instead, they keep only enough information to get the next value. When processing large files, reading a file into a list can use huge memory. Reading it line by line with an iterator uses much less memory and often starts faster, because the first line is available right away.

3.5 Iterator Exhaustion And Reuse

Iterator exhaustion means the iterator has reached its end. After that, the iterator stays exhausted. To reuse iteration, you must create a new iterator from the original iterable or recreate the data source. This helps students plan correctly: if data is needed many times, it may be stored; if it is processed once, it may be streamed.

3.6 The Iterator Cycle In Plain Steps

The iterator cycle can be described as a repeated exchange between the consumer and the iterator. First, the consumer asks for a value. In Python, this happens when a loop starts or when next() is called. Second, the iterator prepares the next value using its state and rule. Third, the iterator returns that value to the consumer. Fourth, the consumer processes the value, such as printing it, adding it to a total, or checking a condition. Then the consumer asks for the next value again.

This cycle continues until one of two endings happens. For a finite iterator, the iterator runs out of values and raises StopIteration. The consumer stops in a clean way. For an infinite iterator, the iterator does not stop by itself, so the consumer must stop the cycle by using a break rule, a counter limit, or another control policy. In both cases, the roles stay the same.

The iterator produces values, and the consumer decides when to stop and how to use each value.

This view is useful for debugging. If a loop seems to run forever, students can ask if the iterator is infinite or if the stopping rule is missing. If memory use grows, students can check whether the consumer is collecting values into a list instead of processing them one by one.

4.0 EXAMPLES AND CASE STUDIES

Examples show how iterators work in code, and case studies show why they matter in real work. The examples below are explained in simple words so that students can focus on the idea. In lab practice, students should also run similar code in Python to observe the behavior.

4.1 Example: Finite Iterator Using A Class

Imagine a class called NaturalUpTo. It produces natural numbers from 1 up to a given limit. The class stores the limit and the current number. When next() is called, it checks whether the current number is within the limit. If yes, it returns the current number and increases it. If not, it raises StopIteration. This shows how state and StopIteration work together.

4.2 Example: Infinite Iterator With Controlled Consumption

Consider an iterator that produces even numbers: 0, 2, 4, 6, and so on. The iterator keeps a current value and adds 2 each time. It never raises StopIteration by itself. So the consumer must control the process. A student might take the first ten even numbers by counting, or stop when the value becomes greater than 100. This shows why infinite producers must be paired with clear stopping rules.

4.3 Example: Manual Recreation Of A For Loop

Students can recreate a for loop step by step. First, create an iterator using iter() on an iterable. Next, call next() repeatedly in a loop. Each call returns a value to process. When the iterator runs out, next() raises StopIteration, and the program stops the loop. This manual view helps students understand the internal working of a for loop.

4.4 Case Study: Iterators In Large Data Processing

A common industry task is log processing. Logs can be very large. A program that reads the full log into memory may fail. Instead, programs often iterate through logs line by line. Each line is parsed and used to update counters or build summaries. This is also used in data analysis pipelines when datasets do not fit in memory. The key idea is to keep memory use small and stable.

4.5 Case Study: Iterators In MCA Projects And Simulations

In MCA projects, students often build simulations, data tools, or testing systems. In a simulation, the next state can be computed from the current state. An iterator-like process can generate the next state when asked, and the consumer can stop when a goal is reached. For testing, an infinite iterator can generate many inputs without storing them. Different consumers can apply different limits, which supports reuse and modular design.

4.6 Mini Practice Tasks For Students

In a practice session, students should apply the ideas in small tasks. They should compare a list-based approach with an iterator-based approach and observe how stopping early saves work. They should also notice where StopIteration happens and how state changes after each next() call.

4.7 Case Study: Iterators In Streaming Sensor Data

Consider a sensor that sends a reading every second. A program may need to store the latest reading, compute an average, and alert if the value is too high. The stream can run for hours or days, so the total number of readings is unknown. If the program stored every reading, memory use would grow until the system became unstable.

An iterator-style design solves this. The producer reads one sensor value at a time and provides it. The consumer processes each value right away. The consumer may keep only a small window, such as the last 60 readings, to compute a moving average. It may also stop after a fixed time for an experiment, such as ten minutes. This design keeps memory stable and makes the program responsive.

This case study also shows separation of control from action. The producer's action is to fetch the next reading. The consumer's control is to decide how long the experiment runs and what rules trigger alerts. If the experiment changes, the producer code can stay the same, and only the consumer rules change.

4.8 Practice Guidance For Reports And Viva

In an undergraduate setting, students may be asked to explain iterators in a report or in a viva. A good explanation includes the definition, the protocol, and one practical example. Students should also mention why iterators matter for memory and for infinite data. When writing, a helpful structure is to describe the producer, describe the consumer, and state the stopping rule.

For example, in a lab report a student can describe a program that reads a file line by line, counts how many lines contain a word, and stops at the end. The producer is the file iterator. The consumer is the counting loop. The stopping rule is StopIteration at the end of the file. For an infinite example, a student can describe a counter that produces numbers and a consumer that stops after a fixed count. The stopping rule is the consumer's limit, not StopIteration.

Students should also connect the concept to algorithm design. Many algorithms are easier to build when data is processed step by step. Iterators match this style and help students think about resource limits in a professional way.

5.0 KEYWORDS AND TOUCH VOCABULARY

Keyword	Meaning
Iterator	An object that produces values one at a time
Iterable	An object that can return an iterator
Lazy evaluation	Computing a value only when it is needed
Iterator protocol	The rules using iter() and next()
iter()	Method that returns the iterator object
next()	Method that returns the next value
StopIteration	The signal that ends iteration
Finite iterator	An iterator with a fixed number of values
Infinite iterator	An iterator that can produce unlimited values
State	The current position or memory of an iterator
Producer	The part that generates values
Consumer	The part that uses values
Stream	A continuous flow of data over time
Memory efficiency	Using less memory while processing data